Compile-Time vs. Runtime Bounds Checking in a Stack-Based Virtual Machine

A Conference Paper Roadmap for a Semester Compiler Design Project

---

1. Overview and Motivation

General-purpose and domain-specific virtual machines routinely perform runtime bounds and safety checks (array bounds, stack limits, type/memory validity) to guarantee program safety. These checks have a real, measurable performance cost. An alternative is to perform equivalent checks at compile time, catching what can be proven safe in advance and only leaving unavoidable checks at runtime. This project designs, builds, and empirically evaluates both approaches on a small custom stack-based virtual machine (VM), quantifying the performance-safety tradeoff and identifying which classes of errors compile-time analysis can and cannot catch.

This directly extends the course syllabus's Module 5 topic of **"security checking of virtual machine code"**, and draws on Module 1 (lexical analysis), Module 2 (parsing), Module 4 (intermediate code generation), and Module 6 (code generation, runtime organization) for the implementation pipeline.

1.1 Why This Topic Is Realistic for a Conference Submission

The underlying question — how expensive is runtime bounds checking, and can compile-time analysis reduce it — is already well-studied in large production systems (e.g., WebAssembly runtimes, static cost-analysis tools). This means the topic is **not a novel contribution to the field at large**, and is unlikely to be accepted at flagship programming-languages/compilers venues (e.g., PLDI, CGO, OOPSLA). It is, however, well suited to:

- Student research symposiums and undergraduate/graduate research tracks
- Regional or university-level CS conferences
- Workshops attached to larger conferences (short/position paper tracks)

These venues evaluate correct methodology, sound understanding of compiler theory, and clear empirical execution — not field-wide novelty. This project is scoped to satisfy exactly that bar within a single semester.

2. Research Questions and Contribution

- **RQ1:** What is the measurable runtime overhead of bounds/safety checking in a stack-based VM, relative to unchecked execution?
- **RQ2:** What proportion of runtime checks can be statically eliminated at compile time without compromising safety, for a representative set of test programs?

- **RQ3:** What classes of errors remain undetectable through static analysis alone, and must remain as runtime checks?

Claimed contribution: a small-scale, reproducible empirical study and working implementation quantifying the compile-time/runtime safety-performance tradeoff in a custom VM, framed explicitly as an applied extension of existing bounds-checking overhead literature rather than a claim of new theory.

3. System Design and Architecture

3.1 The Source Language

A small imperative language supporting: variable declarations, arithmetic expressions, arrays, if/else, while loops, and function calls. Kept intentionally minimal so the full pipeline (lexer through code generation) is buildable within the timeline.

3.2 The Stack-Based Virtual Machine

A custom bytecode VM with instructions such as PUSH, POP, ADD, SUB, LOAD, STORE, ALOAD/ASTORE (array access), JMP, CALL/RET. The VM executes instructions sequentially using an operand stack and a separate memory region for variables/arrays — standard runtime organization covered in Module 6.

3.3 Two Compilation Modes

- **Unchecked mode:** the compiler emits bytecode that performs no bounds/type validation before memory or array access — fastest, but unsafe.
- **Runtime-checked mode:** the compiler emits extra CHECK instructions before every array/memory access, validated at execution time — safe, but with per-access overhead.
- **Compile-time-verified mode:** the compiler performs static analysis (e.g., constant-bound array index analysis, loop-bound analysis) to prove safety in advance wherever possible, inserting runtime checks only where safety cannot be statically guaranteed.

3.4 Pipeline Summary

Stage	Syllabus Module	Output
Lexical analysis	Module 1	Token stream
Parsing	Module 2	Parse tree / AST
Semantic analysis	Module 3	Annotated AST, type info
Intermediate code generation	Module 4	Three-address code
Static safety analysis (novel work)	Module 5	Proven-safe / unproven access list
Code generation (3 modes)	Module 6	Unchecked / checked / hybrid bytecode
Execution & measurement	—	Timing, instruction counts, error coverage

4. Experimental Setup and Evaluation Plan

4.1 Benchmark Programs

A set of 8–12 hand-written test programs of increasing complexity: simple array traversal, nested loops with array access, recursive functions, sorting routines (e.g., bubble sort), and matrix-style 2D array operations. Programs should include both statically-provable-safe cases and cases requiring runtime checks, to properly exercise all three compilation modes.

4.2 Metrics

- Execution time (wall-clock, averaged over multiple runs) for each of the three modes
- Instruction count / bytecode size for each mode
- Percentage of runtime checks eliminated by static analysis (compile-time-verified mode vs. fully runtime-checked mode)
- Manual classification of error types each mode can/cannot catch (e.g., constant-index overflow vs. data-dependent index overflow)

4.3 Expected Outcome Shape

Unchecked mode is expected to be fastest but unsafe; fully runtime-checked mode slowest but fully safe; the compile-time-verified hybrid mode is expected to sit between the two, recovering a measurable fraction of the performance loss while retaining safety for statically provable cases. The paper's central figure is a bar/line chart of execution time across the three modes per benchmark, plus a table of static-elimination percentages.

5. Six-to-Eight Week Build and Writing Timeline

Week	Task
1	Finalize source language grammar; design VM instruction set
2	Implement lexer and parser (Modules 1–2)
3	Implement semantic analysis and intermediate code generation (Modules 3–4)
4	Implement the VM (unchecked execution mode) and basic code generator (Module 6)
5	Implement runtime-checked mode; implement static safety analysis pass (Module 5)
6	Implement compile-time-verified hybrid mode; build benchmark suite
7	Run experiments, collect timing/instruction data, produce charts and tables
8	Write paper: Introduction, Related Work, Design, Evaluation, Conclusion; proofread; format to venue template

Note: 6 weeks covers the build; 2 additional weeks are reserved for writing, revision, and formatting. If the semester is shorter, weeks 5–6 (the hybrid static-analysis mode) can be reduced in scope to a single simple analysis (e.g., constant-index-only proof) rather than a full static analyzer.

6. Recommended Paper Structure

Abstract — 150–250 words: problem, approach, key quantitative result.

1. Introduction — Motivate the safety-performance tradeoff in VM execution; state RQ1–RQ3; summarize contribution.

2. Related Work — Cite WebAssembly bounds-checking studies, static cost-analysis papers, and general VM/bytecode design literature (see References).

3. System Design — Language grammar, VM instruction set, three compilation modes, static analysis approach.

4. Implementation — Pipeline details per syllabus module; key design decisions and limitations.

5. Evaluation — Benchmark suite, metrics, results (tables/charts), discussion of what static analysis could/could not eliminate.

6. Limitations — Small benchmark set; simplified language; static analysis scope; single-machine timing measurements.

7. Conclusion and Future Work — Summarize findings; suggest extensions (larger analysis scope, more benchmarks, real hardware VM).

References — Full citation list (see Section 7 below).

7. Target Venue Strategy

- **Tier 1 (best fit):** Undergraduate/graduate student research symposiums at your institution or regional IEEE/ACM student conferences — explicitly designed to evaluate scoped course/thesis projects.

- **Tier 2:** Workshops co-located with larger systems/PL conferences, submitting to the short-paper or poster track rather than the main research track.
- **Tier 3 (stretch, not primary target):** Regional or national-level CS conferences with broad systems/compilers tracks and a history of accepting applied/empirical student work.
- **Avoid:** Flagship venues (PLDI, CGO, OOPSLA, POPL) — the core research question is already well-established there; submissions would very likely be desk-rejected on novelty grounds.

8. Honest Framing of the Contribution

This paper should explicitly present itself as an **applied, small-scale empirical study** that implements and measures an established tradeoff in a new, custom setting — not as a claim of solving an open research problem. A suggested framing sentence for the abstract/introduction:

> *"We present a small-scale implementation and empirical study applying established compile-time safety verification principles to a custom stack-based virtual machine, quantifying the performance-safety tradeoff and the proportion of runtime checks that can be statically eliminated."*

This framing is accurate, defensible under reviewer scrutiny, and matches exactly what the implementation and evaluation plan above actually produce.

9. Related Work / Reference Stack

The following sources are directly relevant and should be cited in the Related Work and Introduction sections. Verify full bibliographic details (venue, year, page numbers) before final submission.

[1] WebAssembly bounds-checking overhead studies — empirical measurements showing bounds-checking overhead ranging from approximately 20% to over 200% depending on benchmark and execution environment, across multiple WASM runtimes.

[2] Static cost analysis for runtime check elimination — research on inferring upper/lower bounds on program execution cost with and without runtime checks, used to formally justify when checks can be safely omitted or must be retained.

[3] Register-based vs. stack-based virtual machine performance studies — comparative benchmarking of VM architectures under JIT and AOT compilation, relevant background for VM design choices in Section 3.2.

[4] GASOL: Gas Analysis and Optimization for Ethereum Smart Contracts — example of a compiler-adjacent tool performing static analysis to reduce runtime cost in a bytecode-executing VM (EVM); useful as a parallel case study in static-analysis-driven optimization of VM execution cost.

[5] Foundational Verification of Smart Contracts through Verified Compilation — discusses peephole optimization and formally grounded compilation for bytecode-executing VMs; relevant methodological parallel for verifying optimizations preserve safety/semantics.

[6] Classical compiler textbook material on intermediate code generation, backpatching, and runtime organization/activation records (per course syllabus Modules 4 and 6) — foundational citations for the implementation pipeline.

[7] General literature on formally verified compiler passes (e.g., CompCert-style verification) — relevant as a discussion point in Related Work regarding the broader space of compile-time safety guarantees.

Before submission: replace the above descriptive entries with fully formatted citations (author, title, venue, year) in the target venue's required citation style (e.g., ACM, IEEE), and add any additional papers found during a dedicated literature review closer to the writing phase.